

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

Tarea - TIA-05 (Unidad 3 - Tarea 1)
Implementación de un sistema de consultas y vistas

- **Tarea en Equipo**
- **Peso: 20%**
- **Práctica. Caso de Estudio**
- **DML - Manipulación de Datos**
- **Proyecto de Aula**

MIEMBROS DEL EQUIPO:

- Líder: SARA PALACIO ZAPATA
- Miembro: JULIÁN VELÁSQUEZ SALAS

Descripción de la Tarea

1. Contextualización

En el desarrollo del proyecto del sistema de Red Social Pascualina, se ha identificado que uno de los principales riesgos en la construcción de sistemas de información no radica únicamente en la implementación técnica, sino en errores arrastrados desde las etapas iniciales del modelado conceptual y lógico. En este mismo orden de ideas, esos errores se pueden evidenciar en el modelo físico y afectar la manipulación de los datos y la construcción de consultas:

- Una base de datos física mal relacionada (referencias entre tablas)
- Sin controles para evitar inconsistencias y embasuramiento de datos

Estos problemas impactan directamente la calidad de la información, generando bases de datos:

- Vulnerables
- Inconsistentes y difíciles de mantener
- Ineficientes y de baja calidad
- Propensas a errores en consultas

Por esta razón, antes de iniciar la implementación física, es obligatorio realizar un proceso de: Revisión, corrección y mejora del modelo físico. Este proceso debe garantizar que la base de datos física sea:

- Coherente y correcta estructuralmente
- Protegida para evitar inconsistencias durante procesos de actualización
- Amigable y flexible para realizar consultas con información de calidad

En este contexto, el reto consiste en tomar el modelo físico previamente construido para el sistema de Red Social Pascualina y transformarlo en un modelo completamente funcional, implementado en un SGBD, para poder realizar operaciones de inserción, modificación, eliminación y consultas a través del lenguaje de manipulación de datos (DML).

2. Propósito de la Tarea

El objetivo de esta tarea es que los estudiantes:

- Detecten errores en su propio modelo físico y corregirlos. Justifiquen mejoras
- Realicen correctamente operaciones DML: INSERT, DELETE, UPDATE y SELECT
- Implementen operaciones DML en un SGBD (PostgreSQL en este caso)
- Implementen de vistas (VIEWS)
- Utilicen el relacionamiento entre tablas (JOIN), Funciones de agregación y consultas con parámetros.
- Verifiquen las propiedades ACID para establecer la calidad del modelo propuesto

1. Diccionario de Datos Definitivo (Mejorado)

En esta tarea se rediseñó completamente el modelo físico de la Red Social Pascualina para alinearlos con los requerimientos de manipulación de datos (DML) especificados en el enunciado. El modelo anterior tenía 30 tablas distribuidas en múltiples grupos funcionales; el modelo mejorado se simplificó a 17 tablas obligatorias que responden directamente a todas las consultas y operaciones solicitadas.

Las mejoras realizadas al modelo físico anterior son:

- Incorporación de las tablas rol y tipo_usuario como catálogos independientes, con FK desde usuario. El modelo anterior no tenía esta separación.
- La tabla usuario ahora tiene codigo_usuario (VARCHAR(20), UNIQUE) además del email, para identificar usuarios con un código legible.
- La tabla perfil tiene un campo JSONB (informacion_perfil) con índice GIN, que almacena intereses, deportes, habilidades y disponibilidad de mentoría — caso de uso Big Data.
- Las tablas de relación N:M (grupo_usuarios, servicio_usuarios, producto_usuarios, evento_usuarios) tienen campos adicionales para calificaciones, comentarios y fechas de interacción.
- La tabla comentario diferencia explícitamente el id_usuario autor de la publicación y el id_usuario_comentario que comenta — requerimiento explícito del enunciado.
- La tabla evento_usuarios diferencia el promotor del evento (tabla evento) de los usuarios participantes, con campos para calificación, asistencia y comentario post-evento.

#	Tabla	Descripción	Claves y restricciones
1	rol	Roles de usuarios en la Red Social	PK: id_rol UK: nombre_rol
2	tipo_usuario	Tipos de usuario para clasificación	PK: id_tipo_usuario UK: nombre
3	servicio_tipo	Tipos de servicio (catálogo)	PK: id_servicio_tipo UK: nombre
4	producto_tipo	Tipos de producto (catálogo)	PK: id_producto_tipo UK: nombre
5	evento_tipo	Tipos de evento (catálogo)	PK: id_evento_tipo UK: nombre
6	usuario	Usuarios de la Red Social	PK: id_usuario UK: codigo, correo FK: rol, tipo
7	perfil	Perfil extendido con JSONB	PK: id_perfil UK: id_usuario FK: id_usuario
8	publicacion	Publicaciones de usuarios	PK: id_publicacion FK: id_usuario
9	comentario	Comentarios a publicaciones	PK: id_comentario FK: id_publicacion, id_usuario_comentario
10	grupo	Grupos creados por usuarios	PK: id_grupo UK: codigo_grupo FK: id_usuario
11	grupo_usuarios	Tabla N:M usuario-grupo	PK: id_grupo_usuario UK: (id_grupo, id_usuario)
12	servicio	Servicios ofrecidos por usuarios	PK: id_servicio UK: codigo_servicio FK: usuario, tipo

13	servicio_usuarios	Tabla N:M consumo de servicios	PK: id_servicio_usuario FK: id_servicio, id_usuario
14	producto	Productos publicados para venta	PK: id_producto UK: codigo_producto FK: usuario, tipo
15	producto_usuarios	Tabla N:M compras de productos	PK: id_producto_usuario UK: numero_transaccion
16	evento	Eventos publicados por usuarios	PK: id_evento UK: codigo_evento FK: usuario, tipo
17	evento_usuarios	Tabla N:M suscripción a eventos	PK: id_evento_usuario UK: (id_evento, id_usuario)

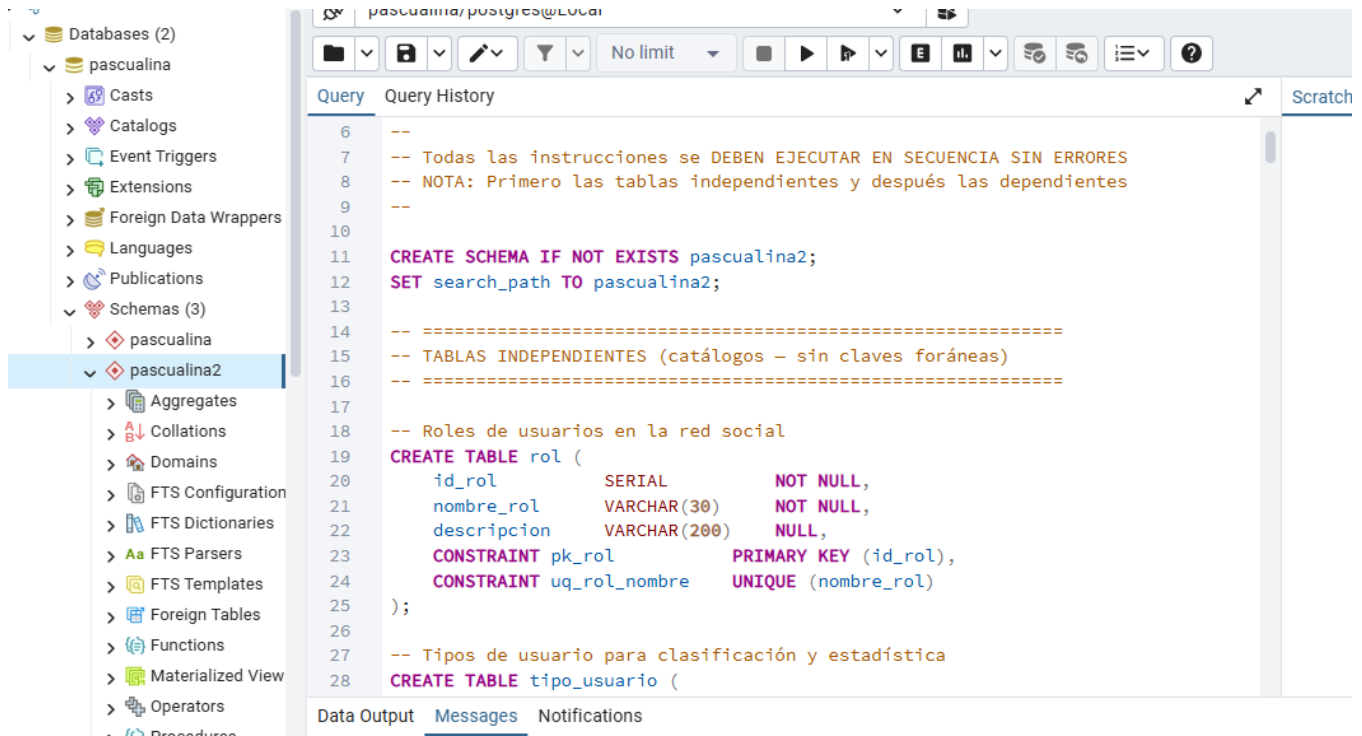
PostgreSQL - Tablas en Detalle										
1	Tabla	rol	Descripción Tabla	Catálogo de roles de los usuarios en la Red Social.					v1.0	
#	Campo	Descripción Campo	Tipo Dato - PostgreSQL	Tamaño	Nulo	PK	FK	UK	Nota/Comentarios	
1	id_rol	Identificador único del rol	SERIAL		No	S			PK autoincremental	
2	nombre_rol	Nombre del rol (administrador, miembro...)	VARCHAR(30)	30	No			S	Valor único	
3	descripcion	Descripción del rol y sus permisos	VARCHAR(200)	200						
2	Tabla	tipo_usuario	Descripción Tabla	Catálogo de tipos de usuario para clasificación y estadística.					v1.0	
#	Campo	Descripción Campo	Tipo Dato - PostgreSQL	Tamaño	Nulo	PK	FK	UK	Nota/Comentarios	
1	id_tipo_usuario	Identificador único del tipo de usuario	SERIAL		No	S			PK autoincremental	
2	nombre_tipo_usuario	Nombre del tipo (estudiante, docente, etc.)	VARCHAR(40)	40	No			S	Valor único	
3	descripcion	Descripción del tipo de usuario	VARCHAR(200)	200						
3	Tabla	servicio_tipo	Descripción Tabla	Catálogo de tipos de servicio. Lo registra el Administrador.					v1.0	
#	Campo	Descripción Campo	Tipo Dato - PostgreSQL	Tamaño	Nulo	PK	FK	UK	Nota/Comentarios	
1	id_servicio_tipo	Identificador único del tipo de servicio	SERIAL		No	S			PK autoincremental	
2	nombre	Nombre del tipo de servicio	VARCHAR(100)	100	No			S	Valor único	
3	descripcion	Descripción del tipo de servicio	VARCHAR(200)	200						

2. Script de Creación de la Base de Datos (CREATE)

El script scripts-02-creacion.sql contiene la creación del esquema pascualina y las 17 tablas en orden estricto: catálogos independientes primero (rol, tipo_usuario, servicio_tipo, producto_tipo, evento_tipo), luego usuario y perfil, y finalmente todas las tablas dependientes.

Cada tabla incluye: clave primaria SERIAL con nombre pk_*, claves foráneas con nombre fk_* y acciones referenciales justificadas, restricciones UNIQUE con nombre uq_*, restricciones CHECK con nombre ck_*, y valores DEFAULT para campos de fecha y estado.

El campo perfil.informacion_perfil es de tipo JSONB con un índice GIN adicional (idx_perfil_jsonb) para optimizar consultas semiestructuradas. Este campo responde al requerimiento de datos Big Data/IoT: en un contexto de producción, los perfiles de usuarios pueden alimentar motores de recomendación de pares sin necesidad de JOINS costosos.



```

6  --
7  -- Todas las instrucciones se DEBEN EJECUTAR EN SECUENCIA SIN ERRORES
8  -- NOTA: Primero las tablas independientes y después las dependientes
9  --
10
11 CREATE SCHEMA IF NOT EXISTS pascualina2;
12 SET search_path TO pascualina2;
13
14 -- =====
15 -- TABLAS INDEPENDIENTES (catálogos – sin claves foráneas)
16 -- =====
17
18 -- Roles de usuarios en la red social
19 CREATE TABLE rol (
20     id_rol          SERIAL          NOT NULL,
21     nombre_rol      VARCHAR(30)    NOT NULL,
22     descripcion     VARCHAR(200)   NULL,
23     CONSTRAINT pk_rol PRIMARY KEY (id_rol),
24     CONSTRAINT uq_rol_nombre UNIQUE (nombre_rol)
25 );
26
27 -- Tipos de usuario para clasificación y estadística
28 CREATE TABLE tipo_usuario (

```

3. Script de Poblamiento (INSERT)

El script scripts-03-poblamiento.sql inserta los registros mínimos obligatorios según los Anexos A, B, C y D del enunciado. Se usó generate_series() de PostgreSQL para generar los 500 usuarios y sus perfiles de forma eficiente.

#	Tabla	Criterio de distribución	Registros
1	rol	4 roles según Anexo A	4
2	tipo_usuario	9 tipos según Anexo A	9
3	servicio_tipo	20 tipos según Anexo B (3 nuevos añadidos)	20
4	producto_tipo	20 tipos según Anexo C (3 nuevos añadidos)	20
5	evento_tipo	20 tipos según Anexo D (3 nuevos añadidos)	20
6	usuario	10 admin + 20 aux + 420 miembro + 50 visitante	500
7	perfil	1 por usuario con JSONB de intereses y habilidades	500
8	publicacion	Generadas por miembros (generate_series)	100
9	comentario	Por usuarios diferentes al autor de la publicación	150
10	grupo	Creados por miembros con código único GRP001...	10
11	grupo_usuarios	Miembros de grupos con ON CONFLICT DO NOTHING	120
12	servicio	50 servicios de 20 tipos diferentes	50
13	servicio_usuarios	Con calificación 1-5 y comentario	150
14	producto	70 productos con precio y stock	70
15	producto_usuarios	30 del último mes + 20 de meses anteriores	50
16	evento	25 pasados + 25 futuros	50
17	evento_usuarios	Con asistencia, calificación y comentario	300

The screenshot shows the PostgreSQL IDE interface. On the left, the 'Databases (2)' tree is expanded to 'pascualina', and then to 'pascualina2'. The main query editor displays a SQL script with the following content:

```

11 SET search_path TO pascualina2;
12
13 -- =====
14 -- 1. ROL (4 registros – Anexo A)
15 -- =====
16 INSERT INTO rol (nombre_rol, descripcion) VALUES
17 ('administrador', 'Tiene acceso a todos los elementos de la base de datos de mane
18 ('auxiliar', 'Tiene acceso al BackOffice de manera limitada'),
19 ('miembro', 'Es miembro de la Red Social. Puede publicar, generar eventos,
20 ('visitante', 'Puede acceder a la Red Social pero no puede interactuar de ning
21
22 -- =====
23 -- 2. TIPO_USUARIO (9 registros – Anexo A)
24 -- =====
25 INSERT INTO tipo_usuario (nombre_tipo_usuario, descripcion) VALUES
26 ('estudiante', 'Estudiante activo de la institución'),
27 ('docente', 'Profesor vinculado a la institución'),
28 ('egresado', 'Graduado de la institución'),
29 ('empleado', 'Personal administrativo u operativo'),
30 ('empresario', 'Empresario aliado de la Red Social'),
31 ('ex_empleado', 'Ex empleado de la institución'),
32 ('ex_estudiante', 'Ex estudiante de la institución'),

```

4. Script de Respaldo (INSERT)

El script scripts-04-respaldo.sql es el respaldo de verificación de la base de datos en formato equivalente a pg_dump. Contiene los INSERTs de los catálogos completos (rol, tipo_usuario, servicio_tipo, evento_tipo) y una muestra representativa de usuarios, perfiles y eventos con ON CONFLICT DO NOTHING para permitir re-ejecución sin errores.

Este script permite al docente restaurar el estado de la base de datos para verificar el correcto funcionamiento de las consultas DML con los datos propios del equipo.

The screenshot shows the PostgreSQL IDE interface. The 'Query' editor displays a SQL script for inserting data into the 'rol' table. The script is as follows:

```

21
22 -- =====
23 -- RESPALDO: rol
24 -- =====
25 INSERT INTO rol (id_rol, nombre_rol, descripcion) VALUES
26 (1, 'administrador', 'Tiene acceso a todos los elementos de la base de datos de mane
27 (2, 'auxiliar', 'Tiene acceso al BackOffice de manera limitada'),
28 (3, 'miembro', 'Es miembro de la Red Social. Puede publicar, generar eventos
29 (4, 'visitante', 'Puede acceder a la Red Social pero no puede interactuar de
30 ON CONFLICT (id_rol) DO NOTHING;
31

```

The 'Data Output' tab shows the following message:

```

INSERT 0 0
Query returned successfully in 74 msec.

```

The status bar at the bottom indicates: Total rows: Query complete 00:00:00.074 CRLF

5. Script de Actualización (UPDATE)

El script scripts-05-update.sql contiene 7 operaciones UPDATE con SELECT de verificación antes y después de cada actualización. Cada operación responde a un escenario de negocio justificado.

- Actualización 1: Dirección del usuario USR0021 (Sara Palacio) — cambio de residencia.
- Actualización 2: Dirección del evento EVT0001 — cambio de sede por razones logísticas.
- Actualización 3: Precio del producto PRD0001 — descuento aplicado por el vendedor.
- Actualización 4: Precio del producto PRD0002 — ajuste por depreciación del artículo.
- Actualización 5: Precio del producto PRD0006 — promoción temporal de ropa.
- Actualización 6: Fecha del evento EVT0003 — reprogramación del Taller SQL Avanzado.
- Actualización 7: Fecha del evento EVT0005 — reprogramación de la Fiesta de Bienvenida.

Query

Query History

1

--

2

-- Scripts de UPDATE de la Base de Datos - SGBD PostgreSQL

3

-- Red Social Estudiantil Pascualina

4

-- Tarea 5 | Grupo: 11 | Equipo: Sara Palacio Zapata, Julián Velásquez Salas

5

-- SGBD: PostgreSQL

6

--

7

8

SET search_path TO pascualina2;

9

10

-- =====

11

-- 1. Actualización de dirección de un usuario

12

-- =====

13

-- Pregunta de negocio: ¿Cómo se actualiza la dirección de un usuario

14

-- que cambió de residencia?

15

UPDATE usuario

16

SET direccion = 'Cra 43A # 18-10 Apt 302, El Poblado, Medellín'

17

WHERE codigo_usuario = 'USR0021';

18

19

-- Verificación

20

SELECT id_usuario, codigo_usuario, nombres, apellidos, direccion

21

FROM usuario

22

WHERE codigo_usuario = 'USR0021';

23

24

-- =====

25

-- 2. Actualización de dirección de un evento

26

-- =====

27

-- Pregunta de negocio: ¿Cómo se actualiza el lugar de un evento

28

-- que cambió de sede por razones logísticas?

29

UPDATE evento

30

SET direccion = 'Centro de Convenciones Plaza Mayor, Medellín'

31

WHERE codigo_evento = 'EVT0001';

32

33

-- Verificación

34

SELECT id_evento, codigo_evento, nombre, direccion

Data Output

Messages

Notifications

+

📄

▼

📄

🗑️

📄

📄

📄

SQL

Showing rows: 1 to 1

	id_evento [PK] integer	codigo_evento character varying (20)	nombre character varying (200)	fecha_evento timestamp without time zone
1	5	EVT0005	Fiesta de Bienvenida 20...	2026-07-20 14:00:00

6. Script de Eliminación (DELETE)

El script scripts-05-delete.sql contiene 3 ciclos INSERT+DELETE con verificación antes y después de cada eliminación. En los casos con tablas hijas (evento_usuarios, servicio_usuarios) se elimina primero el detalle antes de la cabecera para respetar las restricciones de clave foránea.

- Ciclo 1: Producto PRD_ERROR_01 — producto ingresado por error que no se comercializará.
- Ciclo 2: Evento EVT_ERROR_01 — evento cancelado antes de su divulgación oficial.
- Ciclo 3: Servicio SRV_ERROR_01 — servicio que el usuario decidió no ofrecer en la Red.
-
-

```

8  SET search_path TO pascualina2;
9
10 -- =====
11 -- 1. INSERT + DELETE de un producto (se cometió un error)
12 -- =====
13 -- Contexto: Se publicó por error un producto que no se va
14 -- a comercializar todavía. Se inserta y se elimina.
15
16 INSERT INTO producto (id_usuario, id_producto_tipo, codigo_producto, nombre, descripcion, precio, stock)
17 VALUES (31, 1, 'PRD_ERROR_01', 'Producto Error - No Publicar',
18         'Este producto fue ingresado por error y debe ser eliminado.', 0, 0);
19
20 -- Verificación antes de eliminar
21 SELECT id_producto, codigo_producto, nombre, precio
22 FROM   producto
23 WHERE  codigo_producto = 'PRD_ERROR_01';
24
25 -- Eliminación del producto erróneo
26 DELETE FROM producto
27 WHERE  codigo_producto = 'PRD_ERROR_01';
28
29 -- Verificación después de eliminar (debe retornar 0 filas)
30 SELECT id_producto, codigo_producto, nombre
31 FROM   producto
32 WHERE  codigo_producto = 'PRD_ERROR_01';
33
34 -- =====
35 -- 2. INSERT + DELETE de un evento (se cometió un error)
36 -- =====
37 -- Contexto: Se publicó un evento que fue cancelado antes
38 -- de su divulgación oficial.
39

```

Data Output Messages Notifications

SQL

id_servicio [PK] integer	codigo_servicio character varying (20)	nombre character varying (150)
-----------------------------	---	-----------------------------------

7. Script de Consultas de Listados (SELECT)

El script scripts-07-listados.sql contiene 6 consultas SELECT sin uso de wildcard (*). Todas las columnas están explícitamente nombradas con ALIAS descriptivos.

#	JOINS	Pregunta de negocio	Tablas involucradas
1	0	¿Cuál es el listado de usuarios con fecha de nacimiento y fecha de ingreso?	usuario
2	1	¿Cuáles son todos los eventos en orden de fecha descendiente con el promotor?	evento + usuario
3	2	¿Cuáles son los eventos de un usuario con sus suscriptores?	evento + evento_tipo + evento_usuarios + usuario (×2)
4	3	¿Cuáles son los productos vendidos en el último mes con precio y vendedor?	producto_usuarios + producto + producto_tipo + usuario
5	4	¿Cuáles son los servicios de 'Asesoría académica' consumidos en los últimos 3 meses?	servicio_usuarios + servicio + servicio_tipo + usuario (×2)
6	4	Insight propio: grupos con nuevos miembros en últimos 30 días con creador y rol	grupo + usuario + rol + grupo_usuarios + usuario

Consulta #3 — Detalle

La consulta #3 merece descripción especial por su complejidad. Utiliza 2 JOINS y un alias de tabla (u_promotor, u_suscrito) para diferenciar el usuario que creó el evento del usuario que se suscribió. Esto es posible porque la tabla usuario se referencia dos veces en la misma consulta con alias diferentes — técnica fundamental para relaciones reflexivas y multi-rol.

```
FROM evento e
  INNER JOIN usuario      u_promotor ON u_promotor.id_usuario = e.id_usuario
  INNER JOIN evento_tipo  et         ON et.id_evento_tipo    = e.id_evento_tipo
  INNER JOIN evento_usuarios eu       ON eu.id_evento        = e.id_evento
  INNER JOIN usuario      u_suscrito ON u_suscrito.id_usuario = eu.id_usuario
```

Consulta #5 — Detalle

La consulta #5 filtra por tipo de servicio ('Asesoría académica') y por los últimos 3 meses. Usa 4 JOINS y alias de tabla para diferenciar el usuario vendedor del usuario consumidor del servicio — el mismo patrón de alias múltiples sobre la misma tabla.

```

8
9 SET search_path TO pascualina2;
10
11 -- =====
12 -- CONSULTA #1 — Sin JOIN
13 -- Pregunta de negocio: ¿Cuál es el listado completo de usuarios
14 -- registrados en la Red Social Pascualina con su información
15 -- personal, fecha de nacimiento y fecha de ingreso?
16 -- =====
17 SELECT
18     u.codigo_usuario          AS codigo,
19     u.nombres                 AS nombres,
20     u.apellidos               AS apellidos,
21     u.correo                  AS correo_electronico,
22     u.direccion               AS direccion,
23     u.fecha_nacimiento        AS fecha_nacimiento,
24     u.fecha_registro          AS fecha_hora_ingreso_red,
25     u.activo                  AS cuenta_activa
26 FROM usuario u
27 ORDER BY u.fecha_registro DESC;
28
29 -- =====
30 -- CONSULTA #2 — 1 JOIN
31 -- Pregunta de negocio: ¿Cuál es el listado de todos los eventos
32 -- publicados en la Red Social ordenados por fecha descendiente,
33 -- incluyendo el nombre del usuario que generó cada evento?

```

Data Output Messages Notifications

Showing rows: 1 to 6 Page No: 1 of 1

	fecha_consumo_servicio timestamp without time zone	tipo_servicio character varying (100)	codigo_servicio character varying (20)	nombre_servicio character varying (150)	precio_servicio numeric (10,2)	codigo_vendedor character varying (20)	nombre_vendedor text	codigo_consumidor character varying (20)	nombre_consumidor text	calificacion smallint
1	2026-05-16 09:09:22.66959	Asesoría académica	SRV0020	Asesoría laboral CV y entrevist...	200000.00	USR0051	Valentina Vargas	USR0070	David Flores	
2	2026-05-06 09:09:22.66959	Asesoría académica	SRV0040	Asesoría laboral CV y entrevist...	200000.00	USR0071	Alejandro Rivera	USR0090	Ana Córdoba	
3	2026-04-21 09:09:22.66959	Asesoría académica	SRV0020	Asesoría laboral CV y entrevist...	200000.00	USR0051	Valentina Vargas	USR0120	Ana Córdoba	
4	2026-04-11 09:09:22.66959	Asesoría académica	SRV0040	Asesoría laboral CV y entrevist...	200000.00	USR0071	Alejandro Rivera	USR0140	Simón Medina	
5	2026-03-27 09:09:22.66959	Asesoría académica	SRV0020	Asesoría laboral CV y entrevist...	200000.00	USR0051	Valentina Vargas	USR0170	Simón Medina	
6	2026-03-17 09:09:22.66959	Asesoría académica	SRV0040	Asesoría laboral CV y entrevist...	200000.00	USR0071	Alejandro Rivera	USR0190	David Flores	

8. Script de Consultas con Vistas (VIEW)

El script scripts-09-vista.sql crea la vista vw_detalle_eventos con 5 JOINS, ORDER BY y ALIAS. Sigue la estructura en dos pasos requerida por el enunciado.

Objetivo de la vista

Consolidar el detalle completo de los eventos de la Red Social Pascualina: tipo de evento, datos del evento, usuario promotor (con su rol) y usuarios participantes. La Directiva usa esta vista para análisis de actividad, reportes de participación y planificación de agenda.

Sentencia SQL de creación (sin filtro WHERE)

```
CREATE OR REPLACE VIEW vw_detalle_eventos AS
SELECT et.nombre AS tipo_evento, e.codigo_evento, e.nombre AS nombre_evento,
       e.fecha_evento, u_promotor.codigo_usuario AS codigo_promotor,
       u_promotor.nombres || ' ' || u_promotor.apellidos AS nombre_promotor,
       r.nombre_rol AS rol_promotor, u_participante.codigo_usuario AS codigo_participante,
       u_participante.nombres || ' ' || u_participante.apellidos AS nombre_participante
FROM evento e
  INNER JOIN evento_tipo      et          ON et.id_evento_tipo      = e.id_evento_tipo
  INNER JOIN usuario          u_promotor  ON u_promotor.id_usuario   = e.id_usuario
  INNER JOIN rol              r           ON r.id_rol                = u_promotor.id_rol
  INNER JOIN evento_usuarios eu          ON eu.id_evento            = e.id_evento
  INNER JOIN usuario          u_participante ON u_participante.id_usuario = eu.id_usuario
ORDER BY et.nombre ASC, e.fecha_evento DESC;
```

Sentencia SQL de consulta a la vista

```
SELECT vde.tipo_evento, vde.codigo_evento, vde.nombre_evento,
       vde.fecha_evento, vde.nombre_promotor, vde.rol_promotor,
       vde.codigo_participante, vde.nombre_participante
FROM vw_detalle_eventos vde
WHERE vde.fecha_evento >= NOW() - INTERVAL '3 months'
ORDER BY vde.tipo_evento ASC, vde.fecha_evento DESC;
```

Justificación del potencial analítico

La vista permite que analistas y directivos consulten la participación en eventos sin conocer la estructura interna de las tablas ni reescribir los 5 JOINS en cada consulta. En un contexto de Business Intelligence, esta vista sería la base de un dashboard de actividad de la Red Social. El hecho de que no incluya el filtro WHERE en su definición la hace reutilizable para cualquier rango de fechas — el filtro se aplica al momento de consultar.

Query

Query History

Scratch Pad X

```

31 -- PASO 1: CREAR LA VISTA (SIN TÍTULO de Teclas)
32 CREATE OR REPLACE VIEW vw_detalle_eventos AS
33 SELECT
34     et.nombre AS tipo_evento,
35     et.descripcion AS descripción_tipo,
36     e.codigo_evento AS codigo_evento,
37     e.nombre AS nombre_evento,
38     e.descripcion AS descripción_evento,
39     e.direccion AS lugar_evento,
40     e.fecha_evento AS fecha_evento,
41     e.fecha_registro AS fecha_publicacion,
42     e.activo AS evento_activo,
43     u.promotor.codigo_usuario AS codigo_promotor,
44     u.promotor.nombres || ' ' || u.promotor.apellidos AS nombre_promotor,
45     r.nombre_rol AS rol_promotor,
46     eu.fecha_suscripción AS fecha_suscripción,
47     eu.asistio AS asistio_al_evento,
48     eu.calificacion AS calificacion_evento,
49     eu.comentario AS comentario_evento,
50     u.participante.codigo_usuario AS codigo_participante,
51     u.participante.nombres || ' ' || u.participante.apellidos AS nombre_participante
52 FROM evento e
53     INNER JOIN evento_tipo et ON et.id_evento_tipo = e.id_evento_tipo
54     INNER JOIN usuario u_promotor ON u_promotor.id_usuario = e.id_usuario
55     INNER JOIN rol r ON r.id_rol = u_promotor.id_rol
56     INNER JOIN evento_usuarios eu ON eu.id_evento = e.id_evento
57     INNER JOIN usuario u_participante ON u_participante.id_usuario = eu.id_usuario
58 ORDER BY et.nombre ASC, e.fecha_evento DESC;
59
60 -- Verificación: mostrar estructura de la vista
61 SELECT column_name, data_type
62 FROM information_schema.columns
63 WHERE table_schema = 'pascualina'
64 AND table_name = 'vw_detalle_eventos'
65 ORDER BY ordinal_position;
66
67 -- =====
68 -- PASO 2: USAR LA VISTA con filtro de fechas
69 -- Consulta: listado de actividades de los últimos 3 meses
70 -- ordenado por tipo de evento y fecha descendiente

```

Data Output

Messages

Notifications

Showing rows: 1 to 300

Page No: 1

of 1

◀

▶

	tipo_evento character varying (100)	codigo_evento character varying (20)	nombre_evento character varying (200)	descripcion text	lugar character varying (200)	fecha_evento timestamp without time zone
17	Baile	EVT0023	Taller SQL Avanzado	Descripción del evento número 23. Evento organizado por la comunidad Pascualina	Hub de Innovación	2026-03-17 21:09:22.66959
18	Baile	EVT0023	Taller SQL Avanzado	Descripción del evento número 23. Evento organizado por la comunidad Pascualina	Hub de Innovación	2026-03-17 21:09:22.66959
19	Cine - Pelicula	EVT0033	Feria Tecnológica 2026	Descripción del evento número 33. Evento organizado por la comunidad Pascualina	Hub de Innovación	2026-06-02 21:09:22.66959
20	Cine - Pelicula	EVT0033	Feria Tecnológica 2026	Descripción del evento número 33. Evento organizado por la comunidad Pascualina	Hub de Innovación	2026-06-02 21:09:22.66959
21	Cine - Pelicula	EVT0033	Feria Tecnológica 2026	Descripción del evento número 33. Evento organizado por la comunidad Pascualina	Hub de Innovación	2026-06-02 21:09:22.66959
22	Cine - Pelicula	EVT0033	Feria Tecnológica 2026	Descripción del evento número 33. Evento organizado por la comunidad Pascualina	Hub de Innovación	2026-06-02 21:09:22.66959
23	Cine - Pelicula	EVT0033	Feria Tecnológica 2026	Descripción del evento número 33. Evento organizado por la comunidad Pascualina	Hub de Innovación	2026-06-02 21:09:22.66959

9. Verificación de Propiedades ACID

El script scripts-08-acid.sql verifica las cuatro propiedades ACID de PostgreSQL con escenarios concretos de la Red Social Pascualina.

Propiedad	Descripción	Escenario demostrado
Atomicidad	O todo o nada. BEGIN-ROLLBACK revierte ambas operaciones.	UPDATE dirección usuario + UPDATE dirección evento → ROLLBACK → ningún cambio aplicado.
Consistencia	La BD siempre pasa de un estado válido a otro válido.	3 violaciones: PK duplicada en rol, CHECK de calificación > 5, DELETE con FK activa en rol.
Aislamiento	Transacciones concurrentes no se interfieren (hipotético).	MVCC de PostgreSQL: Sesión A actualiza precio, Sesión B ve el valor original hasta el COMMIT.
Durabilidad	Un COMMIT persiste aunque el servidor se reinicie.	INSERT evento_tipo 'Prueba Durabilidad ACID' → COMMIT → SELECT confirma persistencia.

Atomicidad — Detalle

Se ejecuta una transacción BEGIN que actualiza la dirección del usuario USR0031 y la dirección del evento EVT0001, seguida de un ROLLBACK. Los SELECT antes y después del ROLLBACK muestran que ninguno de los dos registros fue modificado — ambas operaciones fueron revertidas como una unidad atómica.

Consistencia — Detalle

Se demuestran tres violaciones de restricciones: (1) INSERT con id_rol=1 ya existente → falla por PK duplicada; (2) UPDATE de calificación con valor 10 → falla por CHECK (calificacion BETWEEN 1 AND 5); (3) DELETE del rol con id=3 que tiene 420 usuarios asignados → falla por ON DELETE RESTRICT en la FK fk_usuario_rol. En todos los casos la base de datos mantiene su estado válido.

Durabilidad — Detalle

Se ejecuta BEGIN → INSERT de un nuevo tipo de evento 'Prueba Durabilidad ACID' → COMMIT. El SELECT posterior confirma que el registro existe y persiste permanentemente. A diferencia del ROLLBACK del escenario de Atomicidad, aquí el COMMIT garantiza que el dato sobrevivirá incluso ante reinicio del servidor.

```

1  --
2  -- Script de Verificación de Propiedades ACID - SGBD PostgreSQL
3  -- Red Social Estudiantil Pascualina
4  -- Tarea 5 | Grupo: 11 | Equipo: Sara Palacio Zapata, Julián Velásquez Salas
5  -- SGBD: PostgreSQL
6  --
7  --
8  SET search_path TO pascualina2;
9  --
10 -- =====
11 -- PROPIEDAD 1: ATOMICIDAD
12 -- Definición: Una transacción se ejecuta completamente
13 -- o no se ejecuta en absoluto. "O todo o nada."
14 --
15 -- Escenario: Un usuario actualiza su dirección y al mismo
16 -- tiempo se registra un nuevo evento. Se usa ROLLBACK para
17 -- demostrar que NINGUNA de las dos operaciones se aplicó.
18 -- =====
19 --
20 -- PASO 1: Ver estado ANTES de la transacción
21 SELECT id_usuario, codigo_usuario, nombres, direccion
22 FROM   pascualina2.usuario
23 WHERE  codigo_usuario = 'USR0031';

```

id_evento_tipo [PK] integer	nombre character varying (100)	descripcion character varying (200)
21	Prueba Durabilidad ACID	Registro creado para verificar la propiedad de Durabilidad A...

10. Análisis de Resultados

La implementación del sistema de consultas DML para la Red Social Estudiantil Pascualina representó el paso más concreto y exigente del curso hasta el momento. A diferencia de las tareas anteriores, donde el trabajo consistía en diseñar estructuras (DDL), esta tarea requirió interactuar con los datos reales almacenados en PostgreSQL: insertarlos, modificarlos, eliminarlos y consultarlos con criterios de negocio precisos. Ese cambio de perspectiva — de diseñador a operador de la base de datos — es fundamental en la formación de un desarrollador de software.

El proceso de mejora del modelo físico fue la primera lección concreta. Las tablas del modelo anterior, aunque correctas estructuralmente, no tenían la granularidad necesaria para responder a todas las consultas solicitadas. La separación de rol y tipo_usuario como catálogos independientes, la adición de campos calificacion y comentario en las tablas de relación N:M, y la diferenciación explícita entre el promotor de un evento y sus participantes son mejoras que parecen pequeñas en papel pero tienen un impacto significativo en la capacidad analítica del sistema.

Las consultas SELECT con múltiples JOINS fueron el ejercicio técnico más valioso. Aprender a referenciar la misma tabla con alias diferentes (u_promotor, u_suscrito) para manejar relaciones con múltiples roles es un patrón que aparece constantemente en sistemas reales — redes sociales, sistemas de e-commerce, plataformas de mensajería. El hecho de que la consulta #3 use la tabla usuario cuatro veces en la misma sentencia (promotor, tipo de usuario del promotor, suscrito, tipo de usuario del suscrito si se quisiera extender) ilustra la potencia del modelo relacional y la importancia de los alias.

La vista vw_detalle_eventos demostró en la práctica por qué las vistas existen. Sin la vista, cualquier analista que quisiera reportar actividad de eventos tendría que reescribir 5 JOINS en cada consulta, con riesgo de errores y duplicación de lógica. Con la vista, la complejidad queda encapsulada y el analista solo escribe WHERE y ORDER BY según sus necesidades. Este patrón es la base de arquitecturas modernas de Business Intelligence y Data Warehousing.

La verificación ACID mostró que las restricciones del modelo no son obstáculos sino garantías. El hecho de que PostgreSQL rechace automáticamente un DELETE sobre un rol con usuarios activos, o un UPDATE con calificación fuera de rango, o un INSERT con PK duplicada, significa que la base de datos protege activamente la calidad del dato sin depender de la lógica de la aplicación. Esa separación de responsabilidades — restricciones en la BD, lógica en la aplicación — es un principio de diseño robusto que reduce errores y simplifica el mantenimiento.

11. Reflexiones y Conclusiones Individuales

Sara

Esta tarea marcó para mí el punto de inflexión en la comprensión de las bases de datos. En las tareas anteriores, el trabajo era esencialmente de diseño: crear esquemas, definir tipos de datos, establecer relaciones. Todo se podía verificar mirando el árbol de objetos en pgAdmin — ¿están las tablas? ¿Tienen las restricciones correctas? Pero en esta tarea, la pregunta cambió: ¿los datos que hay adentro tienen sentido? ¿Las consultas que escribo regresan lo que deben regresar? Ese cambio de pregunta exige un nivel de comprensión diferente.

El ejercicio de poblar 500 usuarios con sus perfiles, publicaciones, grupos, servicios, productos y eventos me hizo entender el orden de inserción como un problema de dependencias, no solo de sintaxis. Cada INSERT fallido por FK me recordó que el modelo es un sistema vivo — cada tabla depende de otras, y esa dependencia hay que respetarla al operar los datos, no solo al diseñarlos. Usar `generate_series()` de PostgreSQL para generar los 500 usuarios de manera eficiente fue también una lección sobre las capacidades del motor más allá del SQL estándar.

Las consultas SELECT fueron el ejercicio que más tiempo me tomó pero también el más satisfactorio. La consulta #5, con 4 JOINS y dos alias de tabla usuario, me costó varios intentos hasta que entendí que la tabla usuario puede aparecer múltiples veces en la misma consulta siempre que se le dé un alias diferente a cada instancia. Una vez que interioricé ese concepto, las consultas complejas dejaron de parecer magia y empezaron a tener una lógica clara: cada JOIN agrega una perspectiva diferente sobre los mismos datos.

La vista fue el componente que más impacto tuvo en mi visión del rol de las bases de datos en sistemas reales. Entender que una vista no almacena datos sino que encapsula una consulta — y que esa encapsulación tiene valor en sí misma porque simplifica el acceso a información compleja — me conecta directamente con el concepto de abstracción que atraviesa toda la ingeniería de software. Una vista bien diseñada es análoga a una función bien diseñada: oculta complejidad, expone solo lo necesario, y puede ser reutilizada sin conocer su implementación interna.

Desde el punto de vista del trabajo en equipo, coordinar con Julián la construcción de un sistema de consultas coherente — donde cada consulta usa los mismos datos generados en el poblamiento — requirió comunicación explícita sobre los datos de prueba. Esa experiencia de coordinación en torno a datos compartidos es exactamente lo que ocurre en un equipo de desarrollo real cuando varios desarrolladores comparten una misma base de datos de prueba. Me llevo de esta tarea no solo competencias técnicas en DML, sino también una metodología de trabajo colaborativo sobre datos.

Julián

La Tarea 5 fue la primera vez en el curso que sentí que estaba haciendo algo que se parece a un sistema de información real. Insertar datos, consultarlos, modificarlos, verificar que las restricciones funcionen — ese ciclo completo es el ciclo de vida de cualquier aplicación que usa una base de datos relacional. Completar ese ciclo sobre el modelo de Pascualina, con datos coherentes y consultas que responden preguntas de negocio reales, fue un aprendizaje cualitativamente diferente al de las tareas anteriores.

El proceso de mejora del modelo fue para mí la parte más reveladora. Al analizar qué tablas necesitaba para responder a la consulta #3 — listar eventos con sus suscriptores diferenciando al promotor — me di cuenta de que el modelo anterior no tenía una forma clara de hacer esa distinción. La tabla evento tenía `id_usuario` pero no había una forma limpia de separar 'el que publicó' de 'los que asistieron'. Agregar `evento_usuarios` como tabla N:M con campos adicionales (`asistio`, `calificacion`, `comentario`) no solo resolvió el problema de la consulta sino que enriqueció el modelo con información de valor para el negocio.

La verificación ACID me mostró algo que no esperaba: las restricciones de la base de datos son actores activos en la integridad del sistema. Cuando intenté hacer DELETE del rol 'miembro' que tenía 420 usuarios asignados y PostgreSQL me devolvió el error de FK, no estaba experimentando una limitación del motor sino una garantía del diseño. El mensaje de error era en realidad la restricción `fk_usuario_rol` funcionando correctamente. Eso cambia la perspectiva: los errores de restricción no son fallos sino evidencia de que el modelo está bien construido.

La propiedad de Durabilidad fue la más conceptualmente poderosa. El hecho de que un COMMIT en PostgreSQL implique que el dato fue escrito en el write-ahead log antes de confirmarse, garantizando que sobrevivirá incluso a una caída de energía en medio de la escritura, conecta el trabajo de la materia con conceptos de sistemas operativos y arquitectura de computadores que parecían alejados. Las bases de datos no son solo colecciones de tablas — son sistemas que ofrecen garantías formales sobre el comportamiento de los datos ante fallos.

Desde el impacto profesional, esta tarea me convenció de que el dominio del DML es una competencia transversal a casi cualquier rol en el mundo tecnológico. No solo los DBA necesitan saber escribir consultas complejas — los desarrolladores de backend las escriben a diario, los analistas de datos las usan para construir reportes, los ingenieros de datos las necesitan para transformar pipelines. Haber construido un sistema de consultas completo sobre un caso de negocio real, con datos significativos y preguntas justificadas, es la base más sólida que pude construir para esas competencias.

Enlace a repositorio: <https://github.com/sarapalac10/bd1-redsocial-grupo11.git>

The screenshot shows the GitHub repository page for 'bd1-redsocial-grupo11'. The repository is public and has 16 commits. The commit history is as follows:

Commit	Description	Time
Tarea-01	Add project report PDF and YouTube video link for Red Socia...	last month
Tarea-02	Agregar documentos de tia 03 unidad 2	5 days ago
Tarea-03	Agregando informe y enlace de youtube.	5 days ago
Tarea-04	WIP: Add scripts, dictionary and video link files.	1 minute ago
Tarea-05	Añadir carpetas Tarea-02 a Tarea-06 con Informe, Resultados ...	last month
Tarea-06	Añadir carpetas Tarea-02 a Tarea-06 con Informe, Resultados ...	last month
README.md	Update README.md	last week

Below the commit history, there is a section for the README file, which is currently empty.

Enlace de Vídeo: <https://youtu.be/GJCJrgkib28>

Rúbrica de Evaluación

#	Ítem	Peso	Cal.
1	Diccionario de Datos (MEJORADO y DEFINITIVO)	20	
2	Script de creación de la base de datos (CREATE)	50	
3	Script de poblamiento de las tablas especificadas (INSERT)	100	
4	Script de respaldo de las tablas pobladas (INSERT)	50	
5	Script de actualización especificada (UPDATE)	10	
6	Script de eliminación especificada (DELETE)	10	
7	Script de consultas de listados (SELECT-JOIN)	120	
8	Script de consultas con vistas (VIEW)	30	
9	Script de verificación propiedades ACID	60	
10	Análisis de resultados (en equipo - 300 palabras mínimo)	30	
11	Reflexiones y Conclusiones individuales (300 palabras mínimo)	40	
12	Informe: Uso de Plantillas, Estructura, Calidad	20	
13	Participación en Foros de Tarea	40	
14	Video de Sustentación	200	
15	Repositorio GIT	20	
NOTA = xx / 800 =		800	